

Daily Scholar Papers Report — 2026-07-12

2026-07-12

Daily Scholar Papers Report — 2026-07-12

[Download PDF](#)

Window covered: 2026-07-11 → 2026-07-12 (Google Scholar alerts + user-curated self-emails, last 24 h)

Executive Summary

Four Scholar-alert threads landed in the window — one each for Xiao Cheng, Zhenchang Xing, and Gelei Deng, plus one Recommended-articles batch of six — for nine raw candidates total. The user-curated forward queue was empty. Two candidates cleared Stage-1: an arXiv preprint from the Xiao Cheng thread that proposes a neuro-symbolic vulnerability detector wrapped around Lean 4, and an arXiv preprint from the Gelei Deng thread introducing a stealth memory-injection attack + 108-case benchmark against persistent personal agents. Both had full arXiv HTML available and are analysed as Outstanding. The remaining seven were dropped at Stage-1 (one on-topic-for-followed-author but off-thesis empirical requirements-engineering study; four DL-for-smart-contract / DL-for-supply-chain saturated topic; one PACMSE SDN-security-testing tool paper without followed-researcher provenance; one calibration/triage secondary-analysis paper). Preference layer is quiet — `venue::preprint` (posterior 0.75, n=2) provides a mild tailwind that both Proceeds already inherit from followed-researcher provenance; no candidate crossed the 0.30 auto-Skip threshold or needed the preference-promoted override.

Outstanding: 2 · **Keep:** 0 · **Borderline High-Priority:** 0

Highlighted Papers

#	Title	Authors	Venue	Link
1.1	LeanGuard: Neuro-Symbolic Reasoning for Vulnerability Detection	Y. Zhao, H. Chen, L. Lu, Z. Yang, X. Cheng, H. Wang	arXiv 2607.03963 (cs.SE), 2026	arXiv
1.2	When Claws Remember but Do Not Tell: Stealthy Memory Injection in Persistent Personal Agents	Y. Zhang, S. Zhao, J. Zhang, J. Zhang, G. Deng, X. Liu	arXiv 2607.05189 (cs.CR/cs.AI), 2026	arXiv

1. Outstanding

1.1 · NEURO-SYMBOLIC VULN DETECTION · Names “premature discharge of safety obligations” as the failure mode of LLM-only vuln detection and forces role separation — LLM as AST-fact filter, Lean 4 kernel as adjudicator, evidence-weighted final verdict — across five CWE classes

1.1 [LeanGuard: Neuro-Symbolic Reasoning for Vulnerability Detection](#) — Y. Zhao, H. Chen, L. Lu, Z. Yang, X. Cheng, H. Wang — arXiv:2607.03963 (cs.SE), 2026

Venue. arXiv preprint, cs.SE (primary).

Problem. Ask an LLM whether a pointer dereference is safe and it will often produce a fluent justification for “yes” that is not a proof. The authors trace this unreliability to a named mechanism — “premature discharge of safety obligations”: when the same component both interprets the code and issues the verdict, the reasoning appears well-formed but no obligation has actually been discharged. The paper’s thesis is that the remedy is not better prompting but a *role separation*.

Method — LeanGuard, three-tier neuro-symbolic architecture.

- **Neural side (semantic filter over AST-extracted candidate facts).** From the target function’s AST an initial fact set is extracted; an LLM prunes spurious facts and keeps real ones. Critically, the LLM never discharges an obligation and never returns a verdict — its output is a filtered fact list.
- **Symbolic side (Lean 4 verification model).** Surviving facts are compiled into a verification model in Lean 4 — a proof assistant whose kernel accepts a conclusion only when it is formally proved. Every dangerous operation must be matched by a guard that provably covers it in scope; if no guard is present, the obligation stays open rather than being argued away.
- **Evidence-aware adjudicator.** Because functions rarely arrive with full context, the symbolic model is necessarily partial — an unproved obligation is not yet a defect. The adjudicator weighs the symbolic and neural verdicts by evidence quality on each side.

Scope. Instantiated on **five CWE classes**; the abstract explicitly names null dereference, use-after-free, and double free (memory-safety cluster). The evaluation is framed as asking “how far this division of labor can be pushed.”

Formal characterisation (professional prose). For each dangerous operation op the Lean model demands a guard g such that Lean’s kernel proves $g \Rightarrow \text{safe}(op)$ in scope; absent such a proof, op remains a candidate obligation, not a decided verdict. (The abstract does not display a labelled formula; this is a paraphrase, not a reproduction.)

Reusability. Two transfers beyond the paper’s own five-CWE scope:

- **“Premature discharge” as a named failure mode** for any LLM-in-the-loop verification pipeline where the same LLM parses code and returns a boolean verdict — a candidate for the same interpret $\neq$ decide split.
- **Filtered-facts $\rightarrow$ proof-assistant compilation** as a general adapter. Lean 4 is not essential — F^* , Coq, Dafny, or an SMT-backed VC generator can be swapped in behind the same fact-filter interface.

Cross-paper connection. Sits naturally alongside the “Automated Static Vulnerability Detection via a Holistic Neuro-symbolic Approach” and “Contextualizing Sink Knowledge for Java Vulnerability Discovery” line covered in earlier digests — LeanGuard is the sharpest articulation yet of why the LLM-as-parser / prover-as-adjudicator split matters.

Closing citable line: “the component that interprets the code must not also be the one that decides a safety obligation is met.”

1.2 · LLM AGENT SECURITY · One-shot black-box email payload trained via env-proxy + rubric-reward RL achieves 87.5% / 71.4% end-to-end stealth memory injection on OpenClaw+GPT-5.4 / Claude Code SDK+Sonnet 4.6, transfers across memory backends and defeats input/model/system-level defences

1.2 [When Claws Remember but Do Not Tell: Stealthy Memory Injection in Persistent Personal Agents](#) — Y. Zhang, S. Zhao, J. Zhang, J. Zhang, G. Deng, X. Liu — arXiv:2607.05189 (cs.CR / cs.AI), 2026 (25 pages, 8 figures)

Venue. arXiv preprint, cs.CR (primary) / cs.AI.

Problem. Persistent personal agents pair long-term memory (files, vector DBs like Mem0) with access to external environments (email, calendars, browsers). Untrusted external content can be silently written into persistent memory and later reused as trusted state. The paper formalises this as **stealth memory injection**: a remote black-box adversary must, with a single email payload, (a) induce the agent to write poisoned memory, (b) stay hidden in the agent’s immediate response to the user, and (c) affect future behaviour — all without runtime feedback.

Contributions.

- **WhisperBench** — a **108-case benchmark** spanning five risk categories and both **fact poisoning** and **preference poisoning**. Built on a real IMAP/SMTP workflow and an authentic email-agent skill, enabling full-cycle evaluation of stealth memory injection attacks.
- **MemGhost** — a one-shot payload generation framework for black-box single-email delivery. An **environment proxy** emulates persistent-agent execution end-to-end; an **objective proxy** converts memory-adoption success and conversational stealth into dense rubric-based rewards. The attacker policy is trained with supervised fine-tuning followed by reinforcement learning against the objective proxy.

Threat model. Remote black-box adversary. Single email payload (no multi-turn / no follow-up). No runtime feedback (no oracle access). Success criteria are jointly (a) memory adoption of the poisoned fact/preference, (b) conversational stealth in the immediate agent reply, and (c) effect on future behaviour.

Headline numbers (verbatim from paper abstract).

- **56 held-out test cases.**
- **87.5% end-to-end success on OpenClaw with GPT-5.4.**
- **71.4% end-to-end success on Claude Code SDK with Sonnet 4.6.**
- Transfers across personal-agent architectures **NanoClaw** and **Hermes Agent**.
- Transfers across memory backends: filesystem and **vector-based Mem0**.
- Remains effective against **input-level, model-level, and system-level defenses**.

Formal characterisation (professional prose). The abstract does not display a labelled formula; the objective proxy is described as a rubric-based reward combining memory-adoption success and conversational-stealth score, optimised via SFT-then-RL.

Reusability. Three transferable ideas:

- **Environment-proxy + objective-proxy training loop** — a black-box attacker training recipe that avoids costly real-agent rollouts by faithfully emulating memory read/write dynamics and scoring against a rubric. Directly portable to tool-use poisoning, calendar poisoning, and browser-history poisoning.
- **WhisperBench structure** — five risk categories × fact/preference split × real IMAP-SMTP infrastructure is a reusable red-team harness template; the 108-case scale and 56-case held-out split are small enough to reproduce inside a lab.
- **Stealth as a first-class metric** — most memory-injection work optimises adoption alone; adding conversational-stealth into the reward penalises “loud” injections that give the user a chance to intervene, which is a cleaner threat model for real-world persistent agents.

Cross-paper connection. Complements the earlier “AI Agents & AI Safety” line in the digest: unlike single-turn prompt-injection benchmarks, WhisperBench requires the payload to survive into memory and re-manifest across sessions — a shift toward evaluating agents rather than raw LLMs.

Closing citable line: “persistent memory can turn ordinary external processing into a practical pathway for long-term agent compromise.”

Cross-Paper Synthesis

Both Outstanding papers hinge on the same architectural instinct: **separate the component that reasons in natural language from the component that decides an obligation or state change**. LeanGuard applies it defensively — LLM ⊥ Lean 4 verdict — so that fluent-but-wrong LLM justifications cannot mask a real defect. MemGhost weaponises the opposite direction — the persistent agent’s memory subsystem *does* decide state changes based on LLM-interpreted external content, and MemGhost exploits precisely that lack of an independent adjudicator to launder external strings into trusted memory. Read together, they define a research programme: any LLM-in-the-loop pipeline that lacks a separately-verified adjudication step is vulnerable to premature-discharge in one direction and stealth-injection in the other. Both papers also lean on **surrogate objectives** — LeanGuard on the Lean-checked guard proof, MemGhost on the environment-proxy + objective-proxy rubric — reinforcing that structured, checkable intermediate signals are the practical currency of LLM-in-loop reliability.

Writing & Rationale Insights

LeanGuard names its failure mode (“premature discharge of safety obligations”) and immediately maps it to an architectural fix (role separation). Naming the failure before naming the tool is unusually effective — the abstract sells the fix as inevitable rather than clever. MemGhost’s abstract front-loads the threat model in one sentence with three success criteria stated conjunctively (write poisoned memory, stay hidden in the reply, affect future behaviour), so any reader can immediately judge whether their defense would break the attack — a strong template for offense papers. Both papers close on a systems-level generalisation (“safety obligation must not be argued away”, “external processing becomes a

compromise pathway”) rather than on a numerical brag, converting an empirical result into a design principle — a useful move for anyone drafting security-adjacent conclusions.